

Performance and productivity enhancements every developer should apply

Performance optimisation relies on measurement, not intuition. Profilers reveal execution patterns, memory usage, and bottlenecks that traditional debugging cannot uncover. Here are some insights.

- CPU profilers help identify expensive functions and tight loops.
- Memory profilers expose object retention, leaks, and inefficient data structures.
- Profiling is especially important for AI-related workloads where speed and resource efficiency matter.

When to use generators, caching, and asynchronous code

Modern Python applications often deal with large datasets, repeated computations, and high-volume network requests. Three powerful tools -- generators, caching, and `async/await` —help optimise performance and make applications more scalable.

Generators: Efficient data processing

Generators allow Python to produce values lazily, rather than loading everything into memory at once. They are ideal when working with streaming data, large files, API pagination, or long-running pipelines.

Use generators when:

- Processing large datasets incrementally
 - Reading files line-by-line
 - Building pipelines where each step feeds into the next
- Here's an example:

```
def read_lines(path):
    for line in open(path):
        yield line.strip()
```

Caching: Speeding up repeated operations

Caching stores the results of expensive functions so that repeated calls return instantly. Python's built-in `functools.cache` and `lru_cache` make this easy and efficient.

Use caching when:

- A function performs repeated heavy computations
- Results depend only on inputs
- You want faster response times without modifying core logic

An example is:

```
from functools import cache
```

```
@cache
def slow_fn(n):
    return n * n # placeholder for heavy work
```

Asynchronous code: High-throughput I/O

Asynchronous programming excels at I/O-bound tasks such as API calls, database queries, and network operations where the program would otherwise spend time waiting.

Use `async` when:

- Your app makes many API/databases calls
 - You need concurrency without threading overhead
 - Building servers, scrapers, chatbots, or automation tools
- An example is:

```
import asyncio
import aiohttp
async def fetch(url):
    async with aiohttp.ClientSession() as s:
        async with s.get(url) as r:
            return await r.text()
```

Python's evolution in recent years has created a development ecosystem that is faster, cleaner, and more maintainable. Developers in 2026 can stand out by understanding not only how to use modern features like structural pattern matching, advanced typing, improved debugging, and smart performance techniques, but also why these matter. By applying these practices, engineers can write clearer code, reduce complexity, and stay competitive in an industry where Python continues to be a central and influential language. **END** 🐧

References

- Python Software Foundation, Python Documentation (v3.13), <https://docs.python.org>
- PEP Index, Python Enhancement Proposals (PEP 634, 695, 712, etc.), <https://peps.python.org>
- Brett Slatkin, Effective Python: 90 Specific Ways to Write Better Python, Addison-Wesley
- Scalene Profiler, High-Performance CPU and Memory Profiling for Python, <https://github.com/plasma-umass/scalene>
- Pyright, Static Type Checker for Python, <https://github.com/microsoft/pyright>

By: Shivam Maurya

The author is a researcher and open source enthusiast.